

Bisqwit's luma-preserving saturation compression algorithm

This algorithm appears originally in [Bisqwit's star field simulation](#) written in QB64. It remaps the color of a pixel in case one or more of its RGB component exceed the value of 1 and it's not overly bright or too dark, i.e. its luminance (brightness) ranges from 0 to 1. This way it better represents color components that would be oversaturated due to the contribution of multiple sources (stars).

Assuming RGB are floats that should (but not always) range between 0 and 1, the algorithm is pretty neat so here are the steps:

Let's have a look in more detail.

RGB to YUV conversion:

$$\begin{bmatrix} Y \\ U \\ V \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.14713 & -0.28886 & 0.436 \\ 0.615 & -0.51499 & -0.10001 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

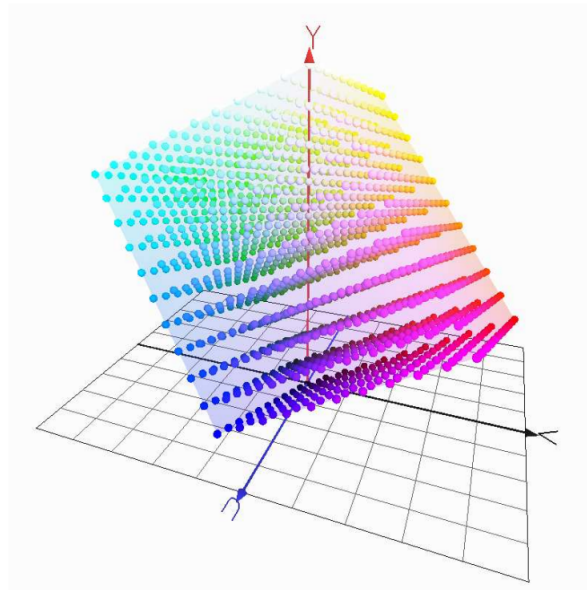


Fig. 1. YUV color space visualization; $Y \in [0, 1]$ and $U, V \in [-0.5, 0.5]$.

Y is the brightness component and unlike many RGB algorithms it does not weigh R, G, B uniformly. U and V are the chrominance (color) components, representing color differences rather than absolute RGB values. U is the blue-projection (or blue-difference) component, and V is the red-projection (or red-difference) component. Because they represent the difference from Y , they are calculated as scaled differences:

$$U \propto (B - Y), \quad V \propto (R - Y)$$

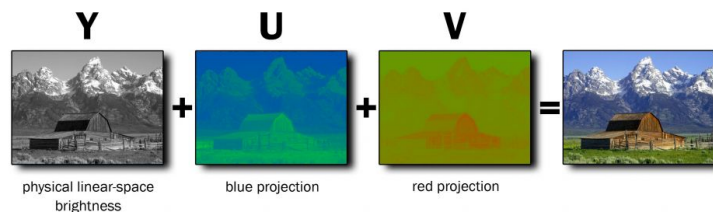


Fig. 2. The components of YUV colorspace.

Luminance is the anchor for saturation control and brightness preservation of colors with acceptable luminance ($Y \in [0, 1]$) but overly/under bright components such that R, G , or $B \notin [0, 1]$. In the latter case, we reconstruct the out-of-bounds value as follows:

$$C' = Y + \text{sat}(C - Y), \quad C = \begin{cases} 1, & C > 1 \\ 0, & C < 0 \end{cases}$$

i.e. to land in the new component C' , we first clamp the old value between 0 and 1 and then linearly interpolate between Y and clamped C . The question is – how to determine sat ?

In case of upper violation ($C > 1$), we require that for the new value:

$$C' = Y + \text{sat}_C(C - Y) \leq 1 \xrightarrow{C-L>0} \text{sat}_C \leq \frac{1 - Y}{C - Y}$$

In the case of lower violation ($C < 0$), we similarly require:

$$C' = Y + \text{sat}_C(C - Y) \geq 0 \xrightarrow{C-L<0} \text{sat}_C \leq \frac{Y}{Y - C}$$

We repeat this for all channels $C \in \{R, G, B\}$ and pick the minimum saturation:

$$C = \min\{1, C_R, C_B, C_G\}$$

The maximum saturation reconstruction is the original color best so we pick the value that corresponds to the equality.

For instance, if $(R, G, B) = (1.4, 0.8, 0.2)$, then $Y = 0.299 \cdot 1.4 + 0.587 \cdot 0.8 + 0.114 \cdot 0.2 \approx 0.911$ and $\text{sat} = \frac{1 - 0.911}{1.4 - 0.911} \approx 0.18$. The reconstructed RGB components are then:

$$\begin{aligned} R' &= Y + 0.18(R - Y) = 0.91 + 0.18(1.4 - 0.91) \approx 1 \\ G' &= Y + 0.18(G - Y) = 0.91 + 0.18(0.8 - 0.91) \approx 0.89 \\ B' &= Y + 0.18(B - Y) = 0.91 + 0.18(0.2 - 0.91) \approx 0.78 \end{aligned}$$

The new luminance is $Y' = 0.299 \cdot 1 + 0.587 \cdot 0.89 + 0.114 \cdot 0.78 \approx 0.91$. So the transform indeed preserves the luminance.

Algorithm 1 Luma-preserving saturation clamp for RGB values

Require: Linear RGB components $R, G, B \in \mathbb{R}$

Ensure: Remapped RGB components $R', G', B' \in [0, 1]$

```
1:  $Y \leftarrow 0.299R + 0.587G + 0.114B$  ▷ Compute luminance
2: if  $Y \geq 1$  then
3:    $R' \leftarrow 1, G' \leftarrow 1, B' \leftarrow 1$ 
4:   return
5: else if  $Y \leq 0$  then
6:    $R' \leftarrow 0, G' \leftarrow 0, B' \leftarrow 0$ 
7:   return
8:  $\text{sat} \leftarrow 1$  ▷ Initialize saturation
9: for  $C \in \{R, G, B\}$  do
10:  if  $C > 1$  then
11:     $\text{sat} \leftarrow \min\left(\text{sat}, \frac{1-Y}{C-Y}\right)$ 
12:  else if  $C < 0$  then
13:     $\text{sat} \leftarrow \min\left(\text{sat}, \frac{Y}{Y-C}\right)$ 
14:  $R' \leftarrow Y + \text{sat}(R - Y)$ 
15:  $G' \leftarrow Y + \text{sat}(G - Y)$ 
16:  $B' \leftarrow Y + \text{sat}(B - Y)$ 
```
